🔵 Java Secure Coding Practices

(Highly Relevant for Senior Software Engineer – Software Security roles)

1️⃣ Input Validation & Sanitization

Never trust external input.

✓ Validate input

- Validate length, format, type
- Use Pattern and Matcher for regex validation
- Reject unexpected characters

✓ Prevent injection attacks

- **SQL Injection** → Use Prepared Statements ONLY
- PreparedStatement ps = conn.prepareStatement("SELECT * FROM users WHERE id = ?");
- ps.setInt(1, id);
- **LDAP Injection** → Use LdapName escape
- **Command Injection** → Avoid Runtime.exec and sanitize arguments
- **No string concatenation in queries**

✓ Use allowlists, not denylists

2️⃣ Output Encoding

Prevents XSS & injection in output layers.

✓ For HTML:

Use:

String safe = StringEscapeUtils.escapeHtml4(userInput);

✓ For URLs:

URLEncoder.encode(value, StandardCharsets.UTF_8);

✓ **For JSON:**

Use a proper serialization library (Jackson/Gson).

3️⃣ Secure Authentication

✓ **Never store plaintext passwords**

Use:

- bcrypt
- Argon2
- PBKDF2

✓ **Use secure session management**

- Generate session IDs using SecureRandom
- Invalidate session on logout
- Set:
- HttpOnly
- Secure
- SameSite=Strict

✓ **Avoid weak JWT signing (HS256 shared secret)**

Use **RS256** with private/public key.

4️⃣ Secure Authorization

✓ **Enforce RBAC / ABAC**

Check role/scope at:

- Controller
- Service layer
- API Gateway

✓ **Never trust client-sent roles/scopes**

Always read from validated JWT claims.

✓ **Enforce object-level authorization**

Prevent:

- IDOR (Insecure Direct Object Reference)

Example:

if (!currentUser.getId().equals(requestedUserId)) throw new ForbiddenException();

5️⃣ **Secure Cryptography**

✓ **Never write your own crypto**

Use:

- javax.crypto
- java.security
- BouncyCastle when needed

✓ **Use strong ciphers**

- AES-256-GCM
- TLS 1.2+
- ECDH key exchange

✓ **Secure random numbers**

SecureRandom sr = new SecureRandom();

Avoid Random.

✓ **Proper key management**

Do not hardcode keys.
Use:

- Vault
- AWS KMS

- GCP KMS

6️⃣ Safe Serialization / Deserialization

✓ Avoid Java native serialization

Can lead to **RCE via gadget chains**.

✓ Prefer safe formats

- JSON
- Jackson
- Gson

✓ If deserialization required:

Use allowlists:

ObjectInputFilter filter = ObjectInputFilter.Config.createFilter("com.app.models.*;!*");

7️⃣ Exception & Error Handling

✓ Don’t expose sensitive info in exceptions

Bad:

SQL Error: username 'admin' not found

Good:

Invalid credentials

✓ Log securely

- Avoid logging passwords/tokens
- Redact PII
- Use centralized logging (ELK, Splunk)

8️⃣ Secure File Handling

✓ **Prevent path traversal**

Paths.get(upload).normalize().toAbsolutePath();

Block:

../../etc/passwd

✓ **Validate file types**

Use MIME detection, not only extension.

✓ **Limit upload size**

Prevent DoS.

9️⃣ **Secure Network Requests**

✓ **Enforce HTTPS**

Never disable SSL checks:

// ❌ Bad

HostnameVerifier hv = (hostname, session) -> true;

✓ **Use timeouts**

HttpClient.newBuilder()

.connectTimeout(Duration.ofSeconds(3));

✓ **Don't trust external services blindly**

Implement:

- retry
- rate limit
- circuit breaker (Resilience4j)

🔟 **Secure Configuration**

✓ **Disable debug modes in production**

✓ **Disable stack traces in client responses**

✓ **Use security headers (server side)**

X-Frame-Options: DENY

X-Content-Type-Options: nosniff

Cache-Control: no-store

✓ **Secure cookies**

Secure; HttpOnly; SameSite=Strict

1️⃣1️⃣ Dependency Security

✓ **Keep dependencies updated**

Most Java security issues come from outdated libraries.

✓ **Use tools:**

- OWASP Dependency-Check
- Snyk
- GitHub Dependabot

✓ **Avoid untrusted libraries (supply chain attacks)**

1️⃣2️⃣ Spring Boot Security Practices

✓ **Use Spring Security**

Don't implement auth manually.

✓ **Never expose Actuator publicly**

If needed:

- Secure with mTLS
- Disable dangerous endpoints: /env, /heapdump

✓ **Validate DTOs**

Use:

@NotNull

@Size(max = 255)

@Email

✓ Use CSRF protection for session-based apps

1️⃣3️⃣ Database Security

✓ Use least privilege DB accounts

- one account per service
- no root or superuser

✓ Encrypt data at rest

Enable:

- TDE
- client-side encryption for sensitive columns

✓ Protect connection strings

Use secret managers, not plaintext configs.

1️⃣4️⃣ Thread Safety & Concurrency Security

✓ Use immutable objects where possible

✓ Avoid shared mutable state

✓ Use synchronized blocks or locks properly

✓ Use Concurrent collections

✓ Avoid Singleton misuse (race conditions)

1️⃣5️⃣ Logging & Monitoring Security

✓ **Log:**

- auth failures
- rate limit events
- suspicious behavior
- privilege escalations

✓ **Don't log:**

- tokens
- passwords
- secrets
- credit cards
- PII

✓ **Use correlation IDs**

Add:

X-Request-ID

🟦 **20-Second Interview Answer**

"Java secure coding means validating all inputs, using prepared statements to prevent SQL injection, encoding outputs, storing passwords with bcrypt/argon2, enforcing RBAC/ABAC, using strong crypto APIs, avoiding unsafe deserialization, handling exceptions securely, protecting files, forcing HTTPS, securing configs, patching dependencies, and using secret management systems instead of hardcoding secrets.
Spring Security, TDE, and Resilience4j help implement this safely."

Java Secure Coding — 1-Page Cheat Sheet

Quick, printable checklist and copy-paste snippets to use during interviews or day-to-day work.

Principles (one-liners)

- **Validate input** (allowlist) → never trust client data.
- **Encode output** before rendering (HTML/URL/JSON).
- **Least privilege** for DB, files, services.
- **Fail safe / deny by default.**
- **Keep secrets out of code** (use Vault/KMS).
- **Use slow salted hashes** (bcrypt / Argon2) for passwords.
- **Prefer secure libs, not DIY crypto.**

Authentication & Passwords

- Use **bcrypt** / **Argon2id**.
- Always salt (auto with bcrypt/argon2) and tune cost (increase with hardware).

```
// bcrypt (BCrypt from spring-security / jBCrypt)
String hash = BCrypt.hashpw(password, BCrypt.gensalt(12));
boolean ok = BCrypt.checkpw(password, hash);
```

- Prefer **OAuth2/OIDC** for SSO; use RS256 for tokens.

Input Validation & Injection Protection

- Use parameterized queries — **never** string concat SQL.

```
PreparedStatement ps = conn.prepareStatement("SELECT * FROM users WHERE id = ?");
ps.setInt(1, id);
```

- For NoSQL/Neo4j: use driver parameter APIs, validate types, whitelist fields.

- Use libraries for CSV/CSV parsing, avoid Runtime.exec or sanitize args.

Output Encoding (prevent XSS)

- HTML: StringEscapeUtils.escapeHtml4(...)
- JSON: use Jackson/Gson serializers (avoid manual building).
- Use CSP header to mitigate inline script risk.

Secure Serialization / Deserialization

- **Avoid native Java serialization**. Prefer JSON (Jackson) with allowed types.
- If unavoidable, **apply an allowlist**:

ObjectInputFilter filter = ObjectInputFilter.Config.createFilter("com.myapp.models.*;!*");

ois.setObjectInputFilter(filter);

Cryptography & Keys

- Use **AES-GCM** for symmetric; use RSA/ECDSA for signatures.
- Use SecureRandom for salts/nonces:

SecureRandom sr = new SecureRandom();

byte[] salt = new byte[16]; sr.nextBytes(salt);

- **Never hardcode keys**. Use KMS/Vault. Use envelope encryption (CMK + DEK).

TLS & Network

- Enforce TLS 1.2/1.3; disable old ciphers.
- Do not disable hostname/certificate validation in clients.
- Use HSTS and HttpsOnly cookies for web apps.

Secrets Management

- Store secrets in **Vault / AWS Secrets Manager / KMS**.
- Use short-lived credentials or dynamic secrets where possible.
- Never commit .env, .properties with secrets.

Secure Session & Cookies

- Cookies: HttpOnly; Secure; SameSite=Strict
- Invalidate sessions on logout.
- CSRF: enable CSRF tokens for cookie-based sessions or use Authorization header (Bearer) for APIs.

Logging & Error Handling

- **Do not log secrets** (passwords, tokens, keys). Mask sensitive fields.
- Return generic error messages to clients; keep stack traces out of responses.
- Log auth failures, rate-limit hits, privilege escalations.

Dependencies & Build

- Keep libs updated. Run automated scans (Snyk, OWASP Dependency-Check).
- Use reproducible builds and sign artifacts where possible.

Concurrency & Resource Handling

- Close DB connections, streams in try-with-resources.
- Set sensible timeouts on HTTP clients.

HttpClient.newBuilder().connectTimeout(Duration.ofSeconds(3));

- Use Circuit Breakers (Resilience4j) + retries + timeouts.

Secure Headers (example NGINX / App)

X-Frame-Options: DENY

X-Content-Type-Options: nosniff

Content-Security-Policy: default-src 'self'

Referrer-Policy: no-referrer

Strict-Transport-Security: max-age=31536000; includeSubDomains

---

Quick Spring Boot Tips

- Use **Spring Security** for auth & CSRF.
- Protect Actuator endpoints (auth + network restrictions).
- Validate DTOs with @Valid, @NotNull, @Size.
- Use property encryption or external secrets for application.properties.

Common Pitfalls to Avoid

- Storing tokens in localStorage (XSS risk) — prefer secure cookies or in-memory storage.
- Using == for string / credential comparisons (use constant-time equals where needed).
- Accepting user-controlled redirects/URLs without validation (open-redirect/SSRF).

Pre-commit Security Checklist

- No secrets in code or commits (git grep for keys).
- Dependencies scanned & updated.
- No use of banned APIs (java serialization / Runtime.exec).
- Input validation present for new endpoints.
- Unit tests for auth/role checks and edge cases.

🔵 What Is Thread-Safe Coding?

Thread-safe code behaves correctly when accessed by multiple threads concurrently — without race conditions, inconsistent data, or visibility issues.

Thread safety =
✓ No race conditions
✓ No data corruption
✓ Correct visibility across threads
✓ Atomic operations where needed

🟦 Why Thread Safety Matters

- Microservices handle thousands of concurrent requests
- Shared objects may be modified by multiple threads
- Race conditions cause unpredictable bugs
- Security bugs (TOCTOU, corrupted state)
- Deadlocks freeze the app
- Data structures become inconsistent

🟩 Java Memory Model (JMM) Basics (Interview-heavy)

Shared variables accessed by multiple threads must be:

- volatile (visibility guarantee) OR
- Guarded by synchronized (visibility + atomicity) OR
- Atomic types (AtomicInteger, AtomicReference) OR
- Immutable objects

Visibility Issue Example:

```
boolean running = true;


void stop() { running = false; }
```

```
void work() {
   while (running) { } // may loop forever (no visibility)
}
```

Fix:

```
volatile boolean running = true;
```

🟧 1. Avoid Shared Mutable State

Best rule:

No shared mutation = no thread safety problems.

Prefer:

- immutable classes
- stateless services
- effectively immutable objects

Example:
String, LocalDate, BigInteger are immutable → always thread-safe.

🟪 2. Use Proper Synchronization

synchronized (monitor lock)

Ensures:

- mutually exclusive access
- visibility of latest values

```
public synchronized void increment() {
   count++;
}
```

ReentrantLock

More flexible:

- tryLock
- fairness
- timed lock

```
lock.lock();
try {
  // critical section
} finally {
  lock.unlock();
}
```

🟥 3. Use High-Level Concurrency Utilities

Java provides **thread-safe** alternatives via java.util.concurrent.

✓ Concurrent Collections:

- ConcurrentHashMap
- ConcurrentLinkedQueue
- CopyOnWriteArrayList
- BlockingQueue

Example:

```
Map<String, String> map = new ConcurrentHashMap<>();
```

✓ Executors:

Use thread pools, don't create threads manually.

```
ExecutorService pool = Executors.newFixedThreadPool(10);
```

🟫 4. Atomic Classes for Lock-Free Thread Safety

Use when updates are simple & fast:

- AtomicInteger
- AtomicReference
- AtomicLong

Example:

```
AtomicInteger counter = new AtomicInteger();
counter.incrementAndGet();
```

🟦 5. Thread-Safe Singleton (Correct Implementation)

Never use a lazy singleton with no synchronization → race condition.

✓ Best Option — Enum Singleton

```
public enum Config {
  INSTANCE;
}
```

✓ Or use Double-Checked Locking with volatile

```
public class Singleton {
  private static volatile Singleton instance;
  public static Singleton getInstance() {
    if (instance == null) {
      synchronized (Singleton.class) {
        if (instance == null) instance = new Singleton();
      }
    }
    return instance;
  }
}
```

🟧 6. Avoid Deadlocks

Deadlock occurs when:

- Thread A holds Lock1 & waits for Lock2
- Thread B holds Lock2 & waits for Lock1

Avoid by:

- Always locking in the same order
- Using tryLock()
- Use timeouts
- Reduce lock scope

🟥 7. Use Thread-Safe Lazy Initialization

Use:

```
class Holder {
  private static class Lazy { static final Service INSTANCE = new Service(); }
  public static Service get() { return Lazy.INSTANCE; } // thread-safe
}
```

🟩 8. Don’t Share Non-Thread-Safe Objects

Examples NOT thread-safe:

- ArrayList
- HashMap
- SimpleDateFormat
- StringBuilder

Use instead:

- CopyOnWriteArrayList
- ConcurrentHashMap

- DateTimeFormatter (immutable)
- StringBuffer (rarely needed)

🟪 9. Minimize Lock Contention

To avoid performance degradation:

- Keep critical sections small
- Use finer-grained locks
- Prefer immutable & lock-free patterns
- Use concurrent collections

🟥 10. Thread Safety in Web Apps / Microservices

Java microservices use:

- thread pools (servlet containers)
- request-per-thread model

Ensure:

- No shared mutable state in controllers
- Services are stateless
- Beans are either immutable or synchronized

Spring Beans are singletons → must be thread-safe.

🟫 11. Common Thread Safety Vulnerabilities

❌ **Race conditions in shared counters**

❌ **Updating shared collections without sync**

❌ **Caching mutable objects**

❌ **Publishing partially constructed objects**

❌ **Using non-thread-safe DateFormat**

❌ **Using double-checked locking without volatile**

❌ **Modifying singleton state unsafely**

🟦 **12. Thread Safety Checklist (1-page)**

- No shared mutable state
- Use immutable objects
- Use synchronized or locks for shared writes
- Use volatile for visibility
- Use concurrent collections
- Use atomic types for simple counters
- Avoid manual thread creation; use ExecutorService
- Protect shared resources (DB connections, caches)
- Ensure Spring beans are stateless
- Avoid deadlocks via lock ordering
- Make Singletons thread-safe
- Use wait/notify correctly or use higher-level APIs

🟩 **20-Second Interview Answer**

“Thread-safe coding means ensuring correctness under concurrency by avoiding shared mutable state, using synchronized blocks or locks for critical sections, using volatile for visibility, and leveraging concurrent collections and atomic classes. Immutable objects and stateless services are inherently thread-safe. Java’s concurrency utilities (Executors, ConcurrentHashMap, ReentrantLock) help avoid race conditions, deadlocks, and visibility issues.”

Why enum singletons are best? Singleton pitfalls in Spring Boot?

ChatGPT said:

Nice — two high-value topics. I'll keep this tight and interview-ready.

✅ Why enum singletons are the best (short answer)

An enum singleton in Java is the simplest, safest way to guarantee *exactly one* instance because the Java language + JVM enforce it for you:

```
public enum MySingleton {
  INSTANCE;

  public void doSomething() { … }
}
```

Key advantages

- **Serialization-safe** — the JVM guarantees a single instance during serialization/deserialization (no need for readResolve()).
- **Reflection-proof** — you cannot create new enum instances via reflection (constructor is private and JVM blocks enum instantiation).
- **Simple & concise** — no volatile, no synchronized, no double-checked locking.
- **Thread-safe by default** — JVM handles safe publication.

When to use

Use enum singletons for *simple, self-contained* singletons that don't need dependency injection (pure utility / global manager with no DI dependencies).

⚠️ When enum singletons are NOT ideal

- You need **constructor injection** (dependencies injected by Spring). Enums can't be instantiated by frameworks with dependencies.
- You need lifecycle callbacks like @PostConstruct / @PreDestroy managed by Spring.

- You want different singleton semantics per environment/classloader (e.g., multiple app contexts in the same JVM).
  In those cases prefer a framework-managed bean.

🔥 **Singleton pitfalls in Spring Boot (what interviewers expect)**

Spring @Singleton (the default bean scope) ≠ JVM-global singleton; it's *container-scoped*. Common pitfalls:

1. Confusing Spring singleton vs JVM singleton

- Spring singleton = one instance *per ApplicationContext*.
- Multiple ApplicationContexts (tests, child contexts, app servers) → multiple instances.

Impact: unexpected duplicate state, resources opened twice, or inconsistent behavior across modules.

2. Static singletons and Spring DI

- Static (manual) singletons (enum or DCL) **bypass Spring**, so you lose DI, lifecycle management, and testability.
- They also complicate unit tests (hard to mock/replace) and can keep resources alive across tests.

Recommendation: prefer Spring-managed beans unless you have a very good reason.

3. Stateful singletons — shared mutable state

- Spring singletons are shared across all threads — **don't store mutable per-request data** in them.
- Race conditions, memory leaks, and security issues (mixing user data) follow.

Fix: make beans stateless or protect mutable state with proper synchronization or use request/prototype scope.

4. Lifecycle & resource cleanup

- If you use manual singletons, you may not get @PreDestroy invoked. Resources (DB connections, threads) may not close.
- Spring beans get lifecycle callbacks; static singletons don't unless you wire them.

5. Testing & isolation problems

- Global singletons make test isolation hard — tests run in same JVM can be affected by leftover state.
- Spring tests can use @DirtiesContext or reset state but it's harder than using DI and mockable beans.

6. Classloader / app server issues

- In app servers or OSGi environments, multiple classloaders can create multiple instances of a static singleton.
- This breaks the "one instance per JVM" assumption.

7. Proxying & AOP interactions

- Spring often returns proxies (CGLIB/JDK proxies) for beans (transactional, security). If you try to use a manual singleton, proxies/advices might not apply.
- Also lifecycle of proxied beans needs Spring control — manual singletons miss that.

8. Serialization / clustering

- If your singleton stores state and the app is clustered, that state may not be shared. Clustering needs distributed caches or external stores (Redis).

Practical recommendations (short checklist)

1. **Prefer Spring-managed singleton beans** for services that need DI, lifecycle, or testing:
2. @Service
3. public class MyService { ... } // default singleton per context
4. **Use enum singleton only** for pure, self-contained global utilities that:
    - have no Spring dependencies, and
    - require JVM-level single-instance + serialization/reflection guarantees.
5. **Keep singletons stateless** or make internal state thread-safe and minimal. Avoid storing request/user data in fields.

6. **If you need per-request data**, use request scope or pass data as method parameters.
7. **For resources needing cleanup**, prefer Spring beans with @PostConstruct/@PreDestroy or implement DisposableBean.
8. **Be careful in tests**: prefer constructor injection + test-specific configs / test slices. Avoid global state in singletons.
9. **If multi-context or classloader environment**, do not assume a single instance across the JVM.

Quick interview-ready answers you can speak

- *"Enum singletons are best because they resist reflection and serialization attacks and are JVM-guaranteed singletons without synchronization complexity."*
- *"In Spring Boot, prefer framework-managed singletons — Spring's 'singleton' means one per ApplicationContext, not one per JVM. Avoid static singletons because they bypass DI, lifecycle, and testability and can cause problems with multiple contexts or classloaders."*